DEVOPS SHACK

LEARN · BUILD · GROW

Troubleshoot Like a Pro

Top 500 Kubernetes Troubleshooting Commands

Diagnose • Fix • Learn • Stay Production Ready

Find Issues

```
$ kubectl get pods
$ kubectl describe pod
$ kubectl logs
$ kubectl exec
$ kubectl get events
  ...
```



Solve Faster

- Identify
- Investigate
- Resolve
- Learn



Better Engineers Build Better Systems

PRACTICAL COMMANDS | REAL-WORLD SCENARIOS | PRODUCTION FOCUSED | BECOME A BETTER ENGINEER

DEVOPS SHACK

LEARN · BUILD · GROW

About This Document

This reference consolidates over 500 practical kubectl, CLI, and operating-system commands used to diagnose and resolve the most common (and several advanced) Kubernetes problems encountered in real production environments. Each command is paired with a concise explanation of when and why to use it, so this document works equally well as a learning resource and as a fast lookup guide during an active incident.

Commands are grouped into 20 categories that mirror the way real troubleshooting unfolds: starting from cluster and node health, moving through pods, workloads, networking, storage, security, and ending with control-plane, Helm, autoscaling, and advanced diagnostics.

How To Use This Reference

Replace placeholders such as <pod-name>, <namespace>, <node-name>, and <service-name> with the actual object names in your cluster. Commands marked as running directly on a node (rather than through kubectl) are explicitly noted in their description.

Published By

This document is brought to you by DevOps Shack — a YouTube channel and learning platform dedicated to practical, hands-on DevOps, Cloud, Kubernetes, and AI/MLOps education. Subscribe at youtube.com/@DevOpsShack and explore more resources at devopsshack.com.

1. Cluster Health & Node Troubleshooting

Commands to verify overall cluster health, inspect node status, capacity, and diagnose node-level issues such as NotReady states, pressure conditions, and kubelet problems.

41 commands in this category

Command	Description
`kubectl cluster-info`	Displays the cluster master and core service endpoints to confirm API server reachability.
`kubectl cluster-info dump`	Dumps detailed cluster state for deep offline analysis and bug reports.
`kubectl get nodes`	Lists all nodes with their status, roles, age, and version.
`kubectl get nodes -o wide`	Lists nodes with additional details like internal/external IP, OS image, and container runtime.
`kubectl describe node <node-name>`	Shows detailed node info including conditions, capacity, allocatable resources, and events.
`kubectl get node <node-name> -o yaml`	Dumps the full YAML spec/status of a node for low-level inspection.
`kubectl top node`	Shows current CPU and memory usage per node (requires metrics-server).
`kubectl top node --sort-by=cpu`	Sorts node resource usage by CPU consumption to spot hot nodes.
`kubectl top node --sort-by=memory`	Sorts node resource usage by memory consumption.
`kubectl get nodes -o json | jq '.items[].status.conditions'`	Extracts node conditions (Ready, MemoryPressure, DiskPressure, PIDPressure) via jq.
`kubectl get events --field-selector involvedObject.kind=Node`	Filters cluster events related specifically to node objects.
`kubectl cordon <node-name>`	Marks a node unschedulable, useful before maintenance or when isolating a faulty node.
`kubectl uncordon <node-name>`	Re-enables scheduling on a node after issues are resolved.
`kubectl drain <node-name> --ignore-daemonsets`	Safely evicts all pods from a node for maintenance while ignoring DaemonSet-managed pods.
`kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data`	Drains a node forcibly including pods using emptyDir volumes.
`kubectl taint nodes <node-name> key=value:NoSchedule`	Applies a taint to prevent further scheduling on a problematic node.
`kubectl taint nodes <node-name> key=value:NoSchedule-`	Removes a previously applied taint from a node.
`kubectl get nodes --show-labels`	Displays all labels on nodes, useful for diagnosing

Command	Description
	scheduling/affinity issues.
`kubectl label node <node-name> key=value`	Adds or updates a label on a node for troubleshooting node selectors.
`kubectl label node <node-name> key-`	Removes a label from a node.
`kubectl get nodes -o custom-columns=NAME:.metadata.name,STATUS:.status.conditions[-1].type`	Custom output showing node name and last reported condition.
`systemctl status kubelet`	Checks kubelet service status directly on a node (run via SSH).
`journalctl -u kubelet -f`	Streams live kubelet logs on a node to debug node agent issues.
`journalctl -u kubelet --since '1 hour ago'`	Reviews recent kubelet logs for errors in the last hour.
`systemctl restart kubelet`	Restarts the kubelet service on a node when it becomes unresponsive.
`crictl ps`	Lists running containers directly via the CRI when kubectl access to a node is limited.
`crictl ps -a`	Lists all containers including stopped ones for deeper container runtime debugging.
`crictl logs <container-id>`	Fetches logs of a specific container using the CRI tool.
`crictl inspect <container-id>`	Shows detailed container runtime metadata for troubleshooting.
`crictl info`	Displays container runtime configuration and status information.
`systemctl status containerd`	Verifies the containerd runtime service health on a node.
`df -h`	Checks node disk usage to diagnose DiskPressure conditions.
`free -m`	Checks node memory usage to diagnose MemoryPressure conditions.
`top`	Displays live node-level process resource consumption.
`kubectl get --raw /healthz`	Queries the raw API server health endpoint.
`kubectl get --raw /healthz?verbose`	Returns verbose health checks for individual API server components.
`kubectl get componentstatuses`	Shows health status of core control plane components (deprecated but still useful in many clusters).
`kubectl get nodes -o jsonpath='{.items[*].status.allocatable}'`	Extracts allocatable CPU/memory/pods per node via JSONPath.
`kubectl get pods --all-namespaces -o`	Lists all pods scheduled on a specific node.

Command	Description
`wide --field-selector spec.nodeName=<node-name>`	
`kubectl get events -A --sort-by=.lastTimestamp`	Lists cluster-wide events sorted chronologically to find recent node/pod failures.
`kubectl version`	Confirms client and server Kubernetes version compatibility, a common source of node issues.

2. Pod Troubleshooting

Core commands for diagnosing pod scheduling failures, crash loops, image pull errors, and runtime issues — the most common day-to-day troubleshooting category.

64 commands in this category

Command	Description
`kubectl get pods`	Lists pods in the current namespace with status summary.
`kubectl get pods -A`	Lists pods across all namespaces.
`kubectl get pods -o wide`	Shows pod IPs, node assignment, and readiness in addition to status.
`kubectl get pods --field-selector=status.phase=Pending`	Filters pods stuck in Pending state.
`kubectl get pods --field-selector=status.phase=Failed`	Filters pods that have failed to run.
`kubectl get pods --field-selector=status.phase=Running`	Filters pods currently running for a health snapshot.
`kubectl describe pod <pod-name>`	Shows full pod details including events, container statuses, and scheduling decisions — the first command for any pod issue.
`kubectl logs <pod-name>`	Retrieves logs for a single-container pod.
`kubectl logs <pod-name> -c <container-name>`	Retrieves logs for a specific container in a multi-container pod.
`kubectl logs <pod-name> --previous`	Retrieves logs from the previous instance of a crashed/restarted container.
`kubectl logs <pod-name> -f`	Streams live logs from a pod.
`kubectl logs <pod-name> --since=10m`	Retrieves logs generated in the last 10 minutes.
`kubectl logs <pod-name> --tail=100`	Retrieves the last 100 lines of pod logs.
`kubectl logs <pod-name> --timestamps`	Includes timestamps with each log line for correlating with events.
`kubectl logs -l app=<label>`	Retrieves logs from all pods matching a label selector.
`kubectl exec -it <pod-name> -- /bin/sh`	Opens an interactive shell session inside a running container.
`kubectl exec -it <pod-name> -c <container-name> -- bash`	Opens a shell in a specific container of a multi-container pod.
`kubectl exec <pod-name> -- env`	Lists environment variables inside a container to debug config issues.
`kubectl exec <pod-name> -- ps aux`	Lists running processes inside a container.

Command	Description
`kubectl exec <pod-name> -- cat /etc/resolv.conf`	Inspects DNS resolver configuration inside a pod.
`kubectl exec <pod-name> -- nslookup <service-name>`	Tests DNS resolution from inside a pod.
`kubectl exec <pod-name> -- curl -v <service-url>`	Tests connectivity to a service endpoint from inside a pod.
`kubectl debug <pod-name> -it --image=busybox`	Attaches an ephemeral debug container to a running pod without restarting it.
`kubectl debug node/<node-name> -it --image=busybox`	Creates a debug pod with access to the host node's namespaces for deep diagnostics.
`kubectl debug <pod-name> --copy-to=debug-pod --container=<name> -it -- sh`	Creates a copy of a pod for safe debugging without affecting the original.
`kubectl get pod <pod-name> -o yaml`	Dumps the full pod manifest including status for manual inspection.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[*].state}'`	Extracts container state (waiting/running/terminated) via JSONPath.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[*].lastState.terminated.reason}'`	Extracts the reason for the last container termination.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[*].restartCount}'`	Shows how many times a container has restarted.
`kubectl delete pod <pod-name>`	Deletes a pod, useful when an owning controller will recreate it cleanly.
`kubectl delete pod <pod-name> --grace-period=0 --force`	Force-deletes a stuck pod that won't terminate normally.
`kubectl get pod <pod-name> --show-labels`	Shows all labels on a pod for verifying service/selector matching.
`kubectl get events --field-selector involvedObject.name=<pod-name>`	Filters events for a specific pod to see scheduling and runtime issues.
`kubectl get events --field-selector reason=Failed`	Filters cluster events with a Failed reason across all objects.
`kubectl get events --field-selector reason=BackOff`	Filters events showing CrashLoopBackOff or ImagePullBackOff occurrences.
`kubectl get events --field-selector reason=FailedScheduling`	Filters events explaining why pods cannot be scheduled.
`kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,RESTARTS:.status.containerStatuses[0].restartCount`	Custom table view to quickly spot unhealthy pods.
`kubectl get pods --sort-by=.status.`	Sorts pods by restart count to identify the most unstable

Command	Description
`containerStatuses[0].restartCount`	workloads.
`kubectl rollout status deployment/<name>`	Checks whether a deployment's pod rollout completed successfully.
`kubectl get rs -l app=<label>`	Lists ReplicaSets backing a deployment to find orphaned or stuck replica sets.
`kubectl get pod <pod-name> -o jsonpath='{.spec.nodeName}'`	Quickly identifies which node a pod is scheduled to.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].image}'`	Lists container images used by a pod to confirm correct versions.
`kubectl get pod <pod-name> -o jsonpath='{.status.podIP}'`	Retrieves the pod's internal cluster IP address.
`kubectl get pod <pod-name> -o jsonpath='{.status.conditions}'`	Shows pod conditions like PodScheduled, Initialized, and Ready.
`kubectl exec <pod-name> -- df -h`	Checks disk usage inside a container, useful for diagnosing volume full errors.
`kubectl exec <pod-name> -- free -m`	Checks memory usage from inside a container.
`kubectl exec <pod-name> -- cat /proc/1/limits`	Inspects resource limits applied to process 1 inside the container's cgroup.
`kubectl cp <pod-name>:/path/to/file ./local-file`	Copies a file out of a container for offline inspection.
`kubectl cp ./local-file <pod-name>:/path/to/file`	Copies a local file into a container, useful for injecting debug scripts.
`kubectl get pod <pod-name> -o jsonpath='{.metadata.ownerReferences}'`	Identifies which controller (Deployment/ReplicaSet/Job) owns a pod.
`kubectl get pod <pod-name> --watch`	Watches a pod's status live for real-time troubleshooting during deploys.
`kubectl get pod <pod-name> -o jsonpath='{.spec.tolerations}'`	Inspects tolerations on a pod to debug taint-related scheduling failures.
`kubectl get pod <pod-name> -o jsonpath='{.spec.affinity}'`	Inspects node/pod affinity rules that may be blocking scheduling.
`kubectl get pod <pod-name> -o jsonpath='{.spec.nodeSelector}'`	Inspects nodeSelector constraints affecting scheduling.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].resources}'`	Reviews requests/limits configured on pod containers.
`kubectl exec <pod-name> -- top`	Runs top inside a container to check live process resource usage.
`kubectl logs <pod-name> --all-containers=true`	Aggregates logs from all containers within a pod in one call.
`kubectl get pod <pod-name> -o jsonpath=`	Shows when a pod actually started, useful for correlating

Command	Description
`'{.status.startTime}'`	with incidents.
`kubectl get pod <pod-name> -o jsonpath='{.status.qosClass}'`	Reveals the pod's QoS class (Guaranteed/Burstable/BestEffort), relevant to eviction order.
`kubectl explain pod.spec.containers.livenessProbe`	Looks up the schema/definition of liveness probe fields directly from the API.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].livenessProbe}'`	Inspects the liveness probe configuration causing restarts.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].readinessProbe}'`	Inspects the readiness probe configuration affecting service traffic.
`kubectl get pod <pod-name> -o jsonpath='{.spec.initContainers[*].name}'`	Lists init containers that may be blocking the main container from starting.
`kubectl logs <pod-name> -c <init-container-name>`	Retrieves logs from a specific init container to debug startup blocking.

3. Deployments, ReplicaSets & Rollouts

Commands for diagnosing failed rollouts, stuck deployments, scaling problems, and ReplicaSet mismatches.

30 commands in this category

Command	Description
`kubectl get deployments`	Lists deployments with desired vs available replica counts.
`kubectl get deployments -o wide`	Adds container image and selector info to deployment listing.
`kubectl describe deployment <name>`	Shows deployment conditions, rollout history, and related events.
`kubectl rollout status deployment/<name>`	Watches rollout progress and reports success or failure.
`kubectl rollout history deployment/<name>`	Lists revision history of a deployment for rollback planning.
`kubectl rollout history deployment/<name> --revision=2`	Shows details of a specific deployment revision.
`kubectl rollout undo deployment/<name>`	Rolls back a deployment to the previous revision.
`kubectl rollout undo deployment/<name> --to-revision=2`	Rolls back to a specific historical revision.
`kubectl rollout pause deployment/<name>`	Pauses an in-progress rollout to investigate issues.
`kubectl rollout resume deployment/<name>`	Resumes a paused rollout.
`kubectl rollout restart deployment/<name>`	Forces a fresh rollout, useful for picking up ConfigMap/Secret changes.
`kubectl scale deployment/<name> --replicas=5`	Manually scales a deployment to a desired replica count for testing.
`kubectl get rs`	Lists all ReplicaSets to find orphaned or excess replica sets.
`kubectl describe rs <name>`	Shows ReplicaSet events and condition details.
`kubectl delete rs <name>`	Removes an orphaned ReplicaSet that is no longer needed.
`kubectl get deployment <name> -o jsonpath='{.status.conditions}'`	Extracts deployment conditions like Available and Progressing.
`kubectl get deployment <name> -o jsonpath='{.spec.strategy}'`	Reviews the deployment's update strategy (RollingUpdate/Recreate).
`kubectl set image deployment/<name> <container>=<image>`	Updates the container image directly to trigger a new rollout.

Command	Description
`kubectl edit deployment <name>`	Opens the deployment manifest in an editor for live troubleshooting edits.
`kubectl get deployment <name> -o yaml | grep -A5 strategy`	Quickly inspects strategy block from raw YAML output.
`kubectl get events --field-selector involvedObject.kind=Deployment`	Filters events specific to Deployment objects.
`kubectl diff -f deployment.yaml`	Shows the difference between a local manifest and the live cluster object before applying.
`kubectl get deployment <name> -o jsonpath='{.status.unavailableReplicas}'`	Shows count of replicas that are currently unavailable.
`kubectl get deployment <name> -o jsonpath='{.status.updatedReplicas}'`	Shows how many replicas have been updated to the latest revision.
`kubectl apply -f deployment.yaml --dry-run=server`	Validates a manifest against the live API server without applying changes.
`kubectl apply -f deployment.yaml --validate=true`	Applies a manifest with strict schema validation enabled.
`kubectl get deployment <name> -o jsonpath='{.spec.template.spec.containers[*].resources}'`	Reviews resource requests/limits defined in the deployment's pod template.
`kubectl autoscale deployment <name> --min=2 --max=10 --cpu-percent=80`	Creates an HPA for a deployment when testing autoscaling behavior.
`kubectl get deployment <name> --show-labels`	Shows labels to confirm correct selector matching with services.
`kubectl patch deployment <name> -p '{"spec":{"replicas":3}}'`	Patches a deployment field directly without editing the full manifest.

4. Services & Networking

Commands for diagnosing service discovery failures, endpoint mismatches, load balancing issues, and cluster networking problems.

50 commands in this category

Command	Description
`kubectl get svc`	Lists services in the current namespace with type and cluster IP.
`kubectl get svc -A`	Lists services across all namespaces.
`kubectl describe svc <name>`	Shows service details including selector, endpoints, and events.
`kubectl get endpoints <name>`	Lists actual pod IPs backing a service — empty output indicates a selector mismatch.
`kubectl get endpoints -A`	Lists endpoints across all namespaces to spot services without backing pods.
`kubectl get svc <name> -o jsonpath='{.spec.selector}'`	Extracts the service's label selector to compare with pod labels.
`kubectl get pods -l <selector> --show-labels`	Verifies which pods actually match a service's selector labels.
`kubectl exec <pod-name> -- curl -v <service-name>.<namespace>.svc.cluster.local`	Tests internal DNS-based service connectivity from within the cluster.
`kubectl exec <pod-name> -- wget -qO- <service-name>:<port>`	Tests service reachability and port availability from inside a pod.
`kubectl get svc <name> -o jsonpath='{.spec.ports}'`	Reviews configured service ports and target ports for mismatches.
`kubectl port-forward svc/<name> 8080:80`	Forwards a local port to a service for direct browser/API testing.
`kubectl port-forward pod/<pod-name> 8080:80`	Forwards a local port directly to a pod, bypassing the service layer for isolation testing.
`kubectl get networkpolicies`	Lists NetworkPolicy objects that may be blocking traffic.
`kubectl describe networkpolicy <name>`	Shows ingress/egress rules of a specific NetworkPolicy.
`kubectl get svc <name> -o yaml`	Dumps full service YAML for manual review of type, ports, and selectors.
`kubectl get ep <name> -o yaml`	Dumps full endpoints YAML showing exact pod IP:port mappings.
`kubectl exec -it <pod-name> -- traceroute <target>`	Traces the network path to a target to localize connectivity failures.
`kubectl exec -it <pod-name> -- ping`	Tests basic ICMP reachability between pods or to

Command	Description
`<target-ip>`	external hosts.
`kubectl exec -it <pod-name> -- netstat -tulnp`	Lists listening ports inside a container to confirm the app is actually bound correctly.
`kubectl exec -it <pod-name> -- ss -tulnp`	Modern alternative to netstat for inspecting socket states inside a container.
`kubectl get svc --field-selector spec.type=LoadBalancer`	Filters for LoadBalancer-type services to check external IP provisioning.
`kubectl get svc <name> -o jsonpath='{.status.loadBalancer.ingress}'`	Checks whether a cloud load balancer has been provisioned and its external address.
`kubectl get events --field-selector involvedObject.kind=Service`	Filters events related specifically to Service objects.
`kubectl get cm kube-proxy -n kube-system -o yaml`	Reviews kube-proxy configuration when service routing misbehaves.
`kubectl logs -n kube-system -l k8s-app=kube-proxy`	Checks kube-proxy logs for iptables/IPVS rule errors.
`kubectl get pods -n kube-system -l k8s-app=kube-proxy -o wide`	Confirms kube-proxy pods are running healthy on every node.
`iptables -L -t nat | grep <service-ip>`	Inspects iptables NAT rules on a node for a specific service IP (run on node).
`ipvsadm -Ln`	Lists IPVS virtual server rules when the cluster uses IPVS proxy mode (run on node).
`kubectl exec <pod-name> -- telnet <service-ip> <port>`	Tests raw TCP connectivity to a service IP and port.
`kubectl get svc <name> -o jsonpath='{.spec.clusterIP}'`	Retrieves the internal ClusterIP assigned to a service.
`kubectl get svc <name> -o jsonpath='{.spec.sessionAffinity}'`	Checks if session affinity (ClientIP) is configured, relevant to sticky session issues.
`kubectl get pods -o wide -l <selector>`	Lists pod IPs matching a selector to manually cross-check against endpoints.
`kubectl exec <pod-name> -- cat /etc/hosts`	Verifies host file entries inside a container affecting name resolution.
`kubectl get svc -o custom-columns=NAME:.metadata.name,TYPE:.spec.type,CLUSTERIP:.spec.clusterIP,PORTS:.spec.ports`	Custom tabular view of services for quick auditing.
`kubectl get endpointslices`	Lists EndpointSlice objects (modern replacement for Endpoints) for large-scale services.
`kubectl describe endpointslice <name>`	Shows detailed EndpointSlice info including readiness per address.
`kubectl get svc <name> -o jsonpath='{.`	Checks external traffic policy (Cluster/Local) affecting

Command	Description
`spec.externalTrafficPolicy}'`	source IP preservation.
`kubectl exec <pod-name> -- curl -I http://<service-name>`	Sends a HEAD request to quickly check HTTP response codes from a service.
`kubectl get pods --selector=app=<name> -o jsonpath='{.items[*].status.podIP}'`	Lists raw pod IPs matching a service selector for manual endpoint comparison.
`kubectl logs -n kube-system -l component=kube-proxy --previous`	Reviews previous kube-proxy container logs after a crash.
`kubectl get cm coredns -n kube-system -o yaml`	Reviews the CoreDNS Corefile configuration affecting cluster DNS networking.
`kubectl exec <pod-name> -- dig <service-name>.<namespace>.svc.cluster.local`	Performs a detailed DNS query to debug service name resolution failures.
`kubectl get svc -A -o jsonpath='{range .items[*]}{.metadata.namespace}{"/"}{.metadata.name}{"\n"}{end}'`	Lists every service namespace/name pair across the cluster.
`kubectl exec <pod-name> -- nc -zv <host> <port>`	Quickly checks TCP port reachability from inside a pod using netcat.
`kubectl get svc <name> -o jsonpath='{.metadata.annotations}'`	Reviews cloud-provider-specific annotations affecting LoadBalancer behavior.
`kubectl proxy --port=8080`	Starts a local proxy to the Kubernetes API server for direct REST debugging.
`curl http://localhost:8080/api/v1/namespaces/default/services`	Queries the API server directly through the kubectl proxy for raw service data.
`kubectl get svc <name> --watch`	Watches a service object live to catch changes during a rolling update.
`kubectl exec <pod-name> -- curl -v --resolve <service-name>:80:<pod-ip> http://<service-name>`	Forces resolution to a specific pod IP to isolate DNS vs network-layer failures.
`kubectl get pod -o jsonpath='{.items[*].spec.dnsPolicy}'`	Checks the DNS policy applied to pods, relevant to custom DNS troubleshooting.

5. Ingress Troubleshooting

Commands for diagnosing ingress controller issues, routing failures, TLS termination problems, and backend misconfigurations.

20 commands in this category

Command	Description
`kubectl get ingress`	Lists ingress resources with hosts and address.
`kubectl get ingress -A`	Lists ingress resources across all namespaces.
`kubectl describe ingress <name>`	Shows ingress rules, backend services, and related events.
`kubectl get ingress <name> -o yaml`	Dumps the full ingress manifest for detailed rule inspection.
`kubectl get pods -n ingress-nginx`	Checks the health of nginx ingress controller pods.
`kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx`	Reviews ingress controller logs for routing or TLS errors.
`kubectl describe svc -n ingress-nginx ingress-nginx-controller`	Inspects the ingress controller's own service exposing it externally.
`kubectl get ingressclass`	Lists available IngressClass objects to confirm correct class assignment.
`kubectl get ingress <name> -o jsonpath='{.spec.rules}'`	Extracts routing rules to verify host/path mappings.
`kubectl get ingress <name> -o jsonpath='{.spec.tls}'`	Checks TLS configuration and referenced secret names on an ingress.
`kubectl get secret <tls-secret-name>`	Confirms the TLS secret referenced by an ingress actually exists.
`kubectl describe secret <tls-secret-name>`	Reviews TLS secret metadata (without exposing key material) for certificate issues.
`openssl x509 -in tls.crt -noout -dates`	Checks expiry dates of a certificate extracted from a TLS secret.
`kubectl exec -n ingress-nginx <controller-pod> -- cat /etc/nginx/nginx.conf`	Inspects the live generated nginx configuration inside the controller pod.
`kubectl exec -n ingress-nginx <controller-pod> -- nginx -t`	Validates the nginx configuration syntax inside the controller pod.
`kubectl get events -n ingress-nginx`	Reviews events in the ingress controller namespace for reload or sync failures.
`kubectl get ingress <name> -o jsonpath='{.status.loadBalancer.ingress}'`	Checks the external address assigned to an ingress resource.
`curl -v -H 'Host: <hostname>' http://`	Tests ingress routing manually by overriding the Host

Command	Description
`<ingress-ip>/`	header.
`kubectl get cm -n ingress-nginx ingress-nginx-controller`	Reviews the ConfigMap controlling global nginx ingress controller behavior.
`kubectl rollout restart deployment -n ingress-nginx ingress-nginx-controller`	Restarts the ingress controller to pick up new configuration or recover from a stuck state.

6. ConfigMaps & Secrets

Commands for verifying configuration injection, secret mounting, and detecting stale or missing configuration data.

20 commands in this category

Command	Description
`kubectl get configmaps`	Lists ConfigMaps in the current namespace.
`kubectl describe configmap <name>`	Shows ConfigMap data keys and related events.
`kubectl get configmap <name> -o yaml`	Dumps the full ConfigMap content for review.
`kubectl get secrets`	Lists secrets in the current namespace.
`kubectl describe secret <name>`	Shows secret metadata and type without revealing values.
`kubectl get secret <name> -o jsonpath='{.data}'`	Retrieves base64-encoded secret data for manual decoding.
`echo '<base64-value>' | base64 --decode`	Decodes a base64-encoded secret value for inspection.
`kubectl get secret <name> -o jsonpath='{.data.<key>}' | base64 -d`	Decodes a specific key from a secret in a single command.
`kubectl exec <pod-name> -- env | grep <VAR_NAME>`	Confirms whether a ConfigMap/Secret-derived environment variable is correctly injected.
`kubectl exec <pod-name> -- ls /etc/config`	Lists mounted ConfigMap files inside a container's volume mount path.
`kubectl exec <pod-name> -- cat /etc/config/<file>`	Reads a mounted ConfigMap file's content directly inside the container.
`kubectl get pod <pod-name> -o jsonpath='{.spec.volumes}'`	Reviews volume definitions to confirm ConfigMap/Secret references are correct.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].volumeMounts}'`	Reviews volume mount paths to ensure config files land where the app expects.
`kubectl create configmap <name> --from-file=<path> --dry-run=client -o yaml`	Previews a ConfigMap manifest before applying it, to catch formatting errors early.
`kubectl rollout restart deployment/<name>`	Forces pods to reload after a referenced ConfigMap/Secret changes, since Kubernetes doesn't auto-restart on config edits.
`kubectl diff -f configmap.yaml`	Shows differences between local and live ConfigMap before applying changes.
`kubectl get events --field-selector involvedObject.kind=Secret`	Filters events related to Secret objects, such as mount failures.
`kubectl get secret <name> -o jsonpath='{.type}'`	Confirms the secret type (Opaque, kubernetes.io/tls, dockerconfigjson) matches expected usage.

Command	Description
`kubectl create secret generic <name> --from-literal=key=value --dry-run=client -o yaml`	Previews a generic secret manifest before applying for validation.
`kubectl get serviceaccount default -o jsonpath='{.secrets}'`	Checks which secrets are auto-associated with a service account, relevant for image pull issues.

7. Storage: PV, PVC & StorageClass

Commands for diagnosing volume binding failures, mount errors, and storage capacity problems.

30 commands in this category

Command	Description
`kubectl get pv`	Lists all PersistentVolumes and their status (Available/Bound/Released).
`kubectl get pvc`	Lists PersistentVolumeClaims in the current namespace with bound status.
`kubectl get pvc -A`	Lists PVCs across all namespaces to find cluster-wide storage issues.
`kubectl describe pvc <name>`	Shows PVC events explaining why a claim is Pending or failed to bind.
`kubectl describe pv <name>`	Shows PersistentVolume details including reclaim policy and claim reference.
`kubectl get storageclass`	Lists available StorageClasses and their provisioners.
`kubectl describe storageclass <name>`	Shows StorageClass parameters and the default annotation status.
`kubectl get pvc <name> -o jsonpath='{.status.phase}'`	Quickly checks if a PVC is Bound, Pending, or Lost.
`kubectl get pvc <name> -o jsonpath='{.spec.resources.requests.storage}'`	Checks the requested storage size on a claim.
`kubectl get pv <name> -o jsonpath='{.spec.capacity.storage}'`	Checks the actual capacity of a bound PersistentVolume.
`kubectl get events --field-selector involvedObject.kind=PersistentVolumeClaim`	Filters events specific to PVC provisioning or binding failures.
`kubectl logs -n kube-system -l app=csi-provisioner`	Reviews CSI provisioner logs when dynamic volume provisioning fails.
`kubectl get pods -n kube-system -l app=csi-provisioner`	Confirms the CSI provisioner pods are running and healthy.
`kubectl exec <pod-name> -- mount | grep <mount-path>`	Verifies a volume is actually mounted at the expected path inside the container.
`kubectl exec <pod-name> -- df -h <mount-path>`	Checks used/available space on a specific mounted volume.
`kubectl get pod <pod-name> -o jsonpath='{.spec.volumes[*].persistentVolumeClaim}'`	Confirms which PVC a pod's volume actually references.
`kubectl delete pvc <name> --grace-`	Force-deletes a stuck PVC, used carefully to unblock

Command	Description
`period=0 --force`	storage cleanup.
`kubectl patch pv <name> -p '{"spec": {"persistentVolumeReclaimPolicy":"Retain"}}'`	Changes a PV's reclaim policy to prevent accidental data loss during troubleshooting.
`kubectl get volumeattachments`	Lists VolumeAttachment objects to debug CSI attach/detach issues.
`kubectl describe volumeattachment <name>`	Shows detailed status of a specific volume attachment failure.
`kubectl get events --field-selector reason=FailedMount`	Filters events showing volume mount failures across the cluster.
`kubectl get events --field-selector reason=FailedAttachVolume`	Filters events showing volume attach failures, common with cloud block storage.
`kubectl get pv -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,CLAIM:.spec.claimRef.name`	Custom tabular view linking PVs to their bound claims.
`kubectl get sc -o jsonpath='{.items[?(@.metadata.annotations."storageclass.kubernetes.io/is-default-class"=="true")].metadata.name}'`	Identifies which StorageClass is set as the cluster default.
`kubectl exec <pod-name> -- touch /mnt/data/testfile`	Tests write access on a mounted volume to confirm permission issues.
`kubectl get pod <pod-name> -o jsonpath='{.spec.securityContext.fsGroup}'`	Checks the fsGroup setting affecting volume file ownership permissions.
`kubectl rollout restart statefulset/<name>`	Restarts a StatefulSet to retry volume attachment after a transient failure.
`kubectl get statefulset <name> -o jsonpath='{.spec.volumeClaimTemplates}'`	Reviews the volume claim template used to auto-provision per-replica storage.
`kubectl get pvc -l app=<label>`	Lists PVCs associated with a specific application label.
`kubectl get csidrivers`	Lists registered CSI drivers in the cluster relevant to provisioning behavior.

8. RBAC & Security Troubleshooting

Commands for diagnosing permission denials, service account misconfigurations, and access control issues.

30 commands in this category

Command	Description
`kubectl auth can-i create pods`	Checks if the current user/context can create pods.
`kubectl auth can-i create pods --as=system:serviceaccount:<ns>:<sa>`	Checks permissions for a specific service account.
`kubectl auth can-i '*' '*' --all-namespaces`	Checks for cluster-admin-equivalent access across all namespaces.
`kubectl auth can-i list secrets -n <namespace>`	Checks namespace-scoped secret access, a common security audit check.
`kubectl get roles -n <namespace>`	Lists namespace-scoped Roles.
`kubectl get rolebindings -n <namespace>`	Lists RoleBindings to see which subjects are bound to which roles.
`kubectl get clusterroles`	Lists cluster-wide ClusterRoles.
`kubectl get clusterrolebindings`	Lists ClusterRoleBindings to find overly broad cluster-wide access grants.
`kubectl describe role <name> -n <namespace>`	Shows the exact rules (verbs/resources) granted by a Role.
`kubectl describe rolebinding <name> -n <namespace>`	Shows which subjects (users/groups/service accounts) are bound to a Role.
`kubectl describe clusterrole <name>`	Shows the exact rules granted by a ClusterRole.
`kubectl describe clusterrolebinding <name>`	Shows subjects bound to a ClusterRole cluster-wide.
`kubectl get serviceaccounts -n <namespace>`	Lists service accounts in a namespace.
`kubectl describe serviceaccount <name> -n <namespace>`	Shows secrets and metadata tied to a service account.
`kubectl get pod <pod-name> -o jsonpath='{.spec.serviceAccountName}'`	Confirms which service account a pod is actually running as.
`kubectl create token <serviceaccount-name>`	Generates a short-lived token for a service account to test API access manually.
`kubectl get events --field-selector reason=Forbidden`	Filters events showing Forbidden errors caused by RBAC denials.
`kubectl get clusterrolebinding -o json | jq '.items[] | select(.subjects[]?.name=="<sa-name>")'`	Finds all bindings granting access to a specific subject via jq filtering.

Command	Description
`kubectl get networkpolicy -A`	Lists NetworkPolicies cluster-wide, relevant when diagnosing blocked traffic as a security control.
`kubectl get psp`	Lists legacy PodSecurityPolicies (deprecated, present in older clusters) restricting pod capabilities.
`kubectl get pod <pod-name> -o jsonpath='{.spec.securityContext}'`	Reviews pod-level securityContext settings like runAsUser and privilege escalation flags.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].securityContext}'`	Reviews container-level security context affecting capability restrictions.
`kubectl get namespace <name> --show-labels`	Checks namespace labels relevant to Pod Security Admission level enforcement.
`kubectl get events --field-selector reason=FailedCreate`	Filters events showing creation failures often tied to admission/security webhook rejections.
`kubectl get validatingwebhookconfigurations`	Lists validating admission webhooks that may be rejecting requests.
`kubectl get mutatingwebhookconfigurations`	Lists mutating admission webhooks that may be altering or blocking object creation.
`kubectl describe validatingwebhookconfiguration <name>`	Shows webhook rules and failure policy for a specific validating webhook.
`kubectl logs -n <webhook-namespace> -l app=<webhook-app>`	Reviews logs of a custom admission webhook server rejecting requests.
`kubectl get pod <pod-name> -o jsonpath='{.spec.imagePullSecrets}'`	Confirms image pull secrets are attached when private registry pulls fail with auth errors.
`kubectl get secret -n <namespace> --field-selector type=kubernetes.io/dockerconfigjson`	Lists docker registry credential secrets in a namespace.

9. Logging, Events & Audit

Commands for centralized event review, audit log inspection, and historical troubleshooting across the cluster.

30 commands in this category

Command	Description
`kubectl get events`	Lists recent events in the current namespace.
`kubectl get events -A`	Lists recent events across all namespaces.
`kubectl get events --sort-by=.lastTimestamp`	Sorts events chronologically for incident timeline reconstruction.
`kubectl get events --sort-by=.metadata.creationTimestamp`	Alternative chronological sort using creation timestamp.
`kubectl get events -w`	Watches events live as they occur for real-time troubleshooting.
`kubectl get events --field-selector type=Warning`	Filters only Warning-type events to focus on potential problems.
`kubectl get events --field-selector type=Normal`	Filters Normal events, useful for confirming expected lifecycle actions occurred.
`kubectl get events -o json | jq '.items[] | select(.type=="Warning")'`	Uses jq to extract and format only warning events for scripting.
`kubectl logs -n kube-system -l component=kube-apiserver`	Reviews API server logs (where accessible) for request failures.
`kubectl logs -n kube-system -l component=kube-controller-manager`	Reviews controller manager logs for reconciliation errors.
`kubectl logs -n kube-system -l component=kube-scheduler`	Reviews scheduler logs for scheduling decision failures.
`kubectl logs -n kube-system -l k8s-app=kube-dns`	Reviews legacy kube-dns logs for resolution issues in older clusters.
`kubectl logs -n kube-system -l k8s-app=coredns`	Reviews CoreDNS logs for resolution failures or config errors.
`kubectl get events --field-selector involvedObject.namespace=<namespace>`	Filters all events within a specific namespace.
`kubectl get events --namespace=<namespace> --field-selector involvedObject.kind=Pod`	Filters pod-specific events within one namespace.
`kubectl logs <pod-name> > pod.log 2>&1`	Exports pod logs to a local file for offline analysis or sharing.
`stern <pod-name-pattern>`	Tails logs from multiple matching pods simultaneously using the Stern CLI tool.

Command	Description
`kubectl logs --selector app=<label> --max-log-requests=20`	Increases concurrent log streaming limit when tailing many pods at once.
`cat /var/log/kube-apiserver-audit.log`	Reviews raw audit logs on the control plane node when audit logging is enabled.
`kubectl get events -o yaml | grep -B5 message`	Greps raw event YAML for message context surrounding specific failures.
`kubectl get events --field-selector reason=Killing`	Filters events showing container kill actions, often tied to OOM or probe failures.
`kubectl get events --field-selector reason=Unhealthy`	Filters events showing failed liveness/readiness probe checks.
`kubectl get events --field-selector reason=NodeNotReady`	Filters events flagging node readiness transitions.
`kubectl get events --field-selector reason=Evicted`	Filters events showing pod evictions due to resource pressure.
`kubectl get pods --field-selector status.reason=Evicted`	Lists pods specifically marked as evicted for cleanup or investigation.
`kubectl delete pods --field-selector status.reason=Evicted`	Bulk-deletes evicted pods to clean up namespace clutter.
`kubectl get events --field-selector reason=Preempting`	Filters events showing pod preemption due to higher-priority scheduling.
`kubectl get pod <pod-name> -o jsonpath='{.status.message}'`	Extracts a human-readable pod status message, often explaining failures.
`kubectl get pod <pod-name> -o jsonpath='{.status.reason}'`	Extracts the short status reason code for a pod's current state.
`kubectl get events --field-selector reason=SuccessfulCreate`	Filters confirmation events for successful object creation, useful for verifying controllers reacted correctly.

10. Resource Usage, Metrics & Performance

Commands for diagnosing CPU/memory bottlenecks, OOMKilled events, and resource quota exhaustion.

30 commands in this category

Command	Description
`kubectl top pods`	Shows live CPU and memory usage per pod in the current namespace.
`kubectl top pods -A`	Shows live resource usage for pods across all namespaces.
`kubectl top pods --sort-by=cpu`	Sorts pods by CPU usage to identify the heaviest consumers.
`kubectl top pods --sort-by=memory`	Sorts pods by memory usage to identify the heaviest consumers.
`kubectl top pods --containers`	Breaks down resource usage per container within each pod.
`kubectl describe pod <pod-name> | grep -A5 'Limits|Requests'`	Quickly extracts resource requests/limits from pod description output.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[*].lastState.terminated.reason}'`	Checks if a container was terminated due to OOMKilled.
`kubectl get events --field-selector reason=OOMKilling`	Filters events specifically reporting out-of-memory kills.
`kubectl get resourcequotas -n <namespace>`	Lists ResourceQuotas applied to a namespace.
`kubectl describe resourcequota <name> -n <namespace>`	Shows current usage against quota limits, useful when pod creation is blocked.
`kubectl get limitranges -n <namespace>`	Lists LimitRange objects enforcing default/min/max resource constraints per namespace.
`kubectl describe limitrange <name> -n <namespace>`	Shows the specific default and boundary values enforced by a LimitRange.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].resources.limits}'`	Extracts configured resource limits for a pod's containers.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].resources.requests}'`	Extracts configured resource requests for a pod's containers.
`kubectl get hpa`	Lists HorizontalPodAutoscalers and their current/target metrics.
`kubectl describe hpa <name>`	Shows detailed HPA scaling events and metric source status.

Command	Description
`kubectl get hpa <name> -o jsonpath='{.status.currentCPUUtilizationPercentage}'`	Checks current CPU utilization percentage reported by an HPA.
`kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes`	Queries raw node metrics directly from the metrics-server API.
`kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods`	Queries raw pod metrics directly from the metrics-server API.
`kubectl get apiservice v1beta1.metrics.k8s.io`	Checks whether the metrics-server API service is registered and available.
`kubectl get pods -n kube-system -l k8s-app=metrics-server`	Confirms the metrics-server pods are running, a prerequisite for kubectl top.
`kubectl logs -n kube-system -l k8s-app=metrics-server`	Reviews metrics-server logs when kubectl top returns no data.
`kubectl get vpa`	Lists VerticalPodAutoscaler objects and their recommendations (if VPA is installed).
`kubectl describe vpa <name>`	Shows detailed VPA resource recommendations for a workload.
`kubectl exec <pod-name> -- cat /sys/fs/cgroup/memory/memory.limit_in_bytes`	Checks the actual cgroup memory limit enforced inside a container.
`kubectl exec <pod-name> -- cat /sys/fs/cgroup/memory/memory.usage_in_bytes`	Checks current cgroup memory usage inside a container in real time.
`kubectl get nodes -o jsonpath='{.items[*].status.capacity}'`	Reviews total node capacity to plan for resource exhaustion scenarios.
`kubectl describe quota -n <namespace>`	Alternate command form for reviewing namespace quota consumption.
`kubectl get pod -A -o jsonpath='{range .items[*]}{.metadata.name}{" "}{.status.containerStatuses[0].restartCount}{"\n"}{end}' | sort -k2 -n -r`	Generates a sorted list of all pods by restart count across the cluster.
`kubectl get pods --field-selector=status.phase!=Running -A`	Lists all non-running pods cluster-wide to spot widespread resource-driven failures.

11. DNS Troubleshooting

Commands focused specifically on CoreDNS health, resolution failures, and cluster name resolution debugging.

20 commands in this category

Command	Description
`kubectl get pods -n kube-system -l k8s-app=kube-dns`	Confirms CoreDNS pods are scheduled and running.
`kubectl describe pod -n kube-system -l k8s-app=kube-dns`	Shows CoreDNS pod events for crash or scheduling issues.
`kubectl logs -n kube-system -l k8s-app=kube-dns`	Reviews CoreDNS logs for query errors and forwarding failures.
`kubectl get cm coredns -n kube-system -o yaml`	Inspects the active CoreDNS Corefile for misconfiguration.
`kubectl edit cm coredns -n kube-system`	Edits the CoreDNS configuration directly to fix forwarding or zone issues.
`kubectl rollout restart deployment coredns -n kube-system`	Restarts CoreDNS pods to apply configuration changes or recover from a stuck state.
`kubectl exec -it <pod-name> -- nslookup kubernetes.default`	Tests resolution of the core Kubernetes API service name from inside a pod.
`kubectl exec -it <pod-name> -- nslookup <service-name>.<namespace>.svc.cluster.local`	Tests full FQDN resolution for a specific service.
`kubectl exec -it <pod-name> -- cat /etc/resolv.conf`	Confirms the nameserver and search domains configured inside a pod.
`kubectl get svc -n kube-system kube-dns`	Confirms the kube-dns/CoreDNS service itself has valid endpoints.
`kubectl get endpoints -n kube-system kube-dns`	Confirms CoreDNS pod IPs are correctly registered as service endpoints.
`kubectl exec -it <pod-name> -- dig +short <hostname>`	Performs a concise DNS lookup to test external or internal name resolution.
`kubectl exec -it <pod-name> -- dig @<coredns-ip> <hostname>`	Queries CoreDNS directly by IP, bypassing normal resolv.conf routing for isolation testing.
`kubectl run dnsutils --image=registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3 -it --rm -- sh`	Spins up a dedicated DNS debugging pod with dig/nslookup pre-installed.
`kubectl get pod <pod-name> -o jsonpath='{.spec.dnsConfig}'`	Reviews custom DNS configuration overrides applied to a specific pod.
`kubectl get nodes -o jsonpath='{.items[*].status.addresses}'`	Cross-checks node IPs relevant to upstream DNS forwarding configuration.

Command	Description
`kubectl top pods -n kube-system -l k8s-app=kube-dns`	Checks CoreDNS resource usage, since CPU throttling can cause intermittent resolution failures.
`kubectl get hpa coredns -n kube-system`	Checks if CoreDNS autoscaling is configured and functioning under load.
`kubectl describe cm coredns-custom -n kube-system`	Reviews custom CoreDNS configuration overrides defined separately from the main Corefile.
`kubectl exec -it <pod-name> -- getent hosts <hostname>`	Tests name resolution using the system resolver library directly inside a container.

12. Control Plane & etcd Troubleshooting

Commands for diagnosing API server, controller manager, scheduler, and etcd cluster health on self-managed clusters.

24 commands in this category

Command	Description
`kubectl get pods -n kube-system`	Lists all control plane component pods in a self-managed/kubeadm cluster.
`kubectl get pods -n kube-system -l component=etcd`	Checks etcd pod health and status.
`kubectl logs -n kube-system -l component=etcd`	Reviews etcd logs for leader election or disk I/O issues.
`etcdctl endpoint health`	Checks etcd endpoint health directly via the etcdctl CLI (run on a control plane node).
`etcdctl endpoint status --write-out=table`	Shows detailed etcd status including DB size and leadership in table format.
`etcdctl member list`	Lists all etcd cluster members and their status.
`etcdctl --endpoints=<endpoint> alarm list`	Lists any active etcd alarms such as NOSPACE warnings.
`kubectl get pods -n kube-system -l component=kube-apiserver`	Confirms API server pods are running on control plane nodes.
`kubectl logs -n kube-system -l component=kube-apiserver --tail=200`	Reviews recent API server logs for request handling errors.
`kubectl get pods -n kube-system -l component=kube-controller-manager`	Confirms controller manager pod health.
`kubectl get pods -n kube-system -l component=kube-scheduler`	Confirms scheduler pod health.
`kubectl get leases -n kube-system`	Lists leader election leases to confirm which control plane instance holds leadership.
`kubectl describe lease kube-scheduler -n kube-system`	Shows the current scheduler leader and lease renewal timestamps.
`crictl ps --label io.kubernetes.pod.namespace=kube-system`	Lists control plane containers directly via CRI on a node.
`cat /etc/kubernetes/manifests/kube-apiserver.yaml`	Reviews the static pod manifest controlling API server startup flags (kubeadm clusters).
`systemctl status kubelet -l`	Checks the kubelet status on a control plane node managing static pods.
`kubeadm certs check-expiration`	Checks expiration dates of all cluster certificates in a kubeadm-managed cluster.

Command	Description
`kubeadm certs renew all`	Renews all kubeadm-managed cluster certificates before expiry causes an outage.
`kubectl get csr`	Lists pending CertificateSigningRequests, relevant to kubelet bootstrap issues.
`kubectl certificate approve <csr-name>`	Approves a pending certificate signing request.
`kubectl get --raw /metrics | grep apiserver_request_total`	Pulls raw API server metrics to analyze request volume and error rates.
`kubectl get --raw /apis`	Lists all registered API groups to confirm aggregated API servers are reachable.
`kubectl get apiservices`	Lists all APIService objects and their availability status.
`kubectl describe apiservice <name>`	Shows why a specific aggregated API service is unavailable.

13. Helm Troubleshooting

Commands for diagnosing failed Helm releases, chart rendering issues, and upgrade rollback scenarios.

20 commands in this category

Command	Description
`helm list`	Lists all Helm releases in the current namespace.
`helm list -A`	Lists Helm releases across all namespaces.
`helm status <release-name>`	Shows the current status and resources of a specific release.
`helm get values <release-name>`	Shows the values actually used for a deployed release.
`helm get manifest <release-name>`	Shows the full rendered Kubernetes manifest produced by a release.
`helm get notes <release-name>`	Displays post-install notes that may contain troubleshooting hints from the chart author.
`helm history <release-name>`	Shows revision history of a release for rollback planning.
`helm rollback <release-name> <revision>`	Rolls back a release to a previous working revision.
`helm diff upgrade <release-name> <chart> -f values.yaml`	Shows what an upgrade would change before actually applying it (requires helm-diff plugin).
`helm template <chart> -f values.yaml`	Renders chart templates locally without installing, useful for catching templating errors early.
`helm lint <chart>`	Validates a chart's structure and templates for common errors.
`helm install <release> <chart> --dry-run --debug`	Simulates an install and prints the rendered manifest with debug output.
`helm upgrade <release> <chart> --dry-run --debug`	Simulates an upgrade and prints the rendered manifest with debug output.
`helm uninstall <release-name>`	Removes a broken release cleanly to allow a fresh reinstall.
`helm get hooks <release-name>`	Lists hooks defined for a release, useful when install/upgrade hangs on a hook job.
`kubectl get jobs -l app.kubernetes.io/managed-by=Helm`	Lists Helm-managed Job objects, relevant to stuck pre/post-install hooks.
`helm repo update`	Refreshes local chart repository indexes to fix stale chart version errors.
`helm search repo <chart-name>`	Searches available chart versions in configured repositories.
`kubectl get secrets -l owner=helm`	Lists Helm release storage secrets directly for low-level

Command	Description
	release state inspection.
`helm get all <release-name>`	Combines values, manifest, notes, and hooks output for a comprehensive release dump.

14. CrashLoopBackOff & Pod Lifecycle States

Focused command set for diagnosing the most common pod failure states: CrashLoopBackOff, ImagePullBackOff, Pending, and Terminating.

20 commands in this category

Command	Description
`kubectl describe pod <pod-name> | grep -A10 Events`	Extracts just the events section, usually the fastest path to root cause.
`kubectl logs <pod-name> --previous --tail=50`	Shows the last lines of the previous crashed container instance.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[0].state.waiting.reason}'`	Extracts the specific waiting reason (CrashLoopBackOff, ImagePullBackOff, etc.).
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[0].state.waiting.message}'`	Extracts the detailed waiting message with more context than the reason code.
`kubectl get events --field-selector involvedObject.name=<pod-name>,reason=BackOff`	Filters BackOff-specific events for a single pod.
`kubectl describe pod <pod-name> | grep -i image`	Quickly checks the exact image reference being pulled for typos or tag issues.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].imagePullPolicy}'`	Checks the imagePullPolicy, since Always can cause repeated registry hits and rate-limit errors.
`docker pull <image>`	Manually tests if an image can be pulled outside Kubernetes to isolate registry-side issues.
`kubectl get pod <pod-name> -o jsonpath='{.status.phase}'`	Quickly checks the overall pod phase (Pending, Running, Failed, Succeeded, Unknown).
`kubectl get pod <pod-name> -o jsonpath='{.metadata.deletionTimestamp}'`	Checks if a pod is stuck Terminating by inspecting its deletion timestamp.
`kubectl get pod <pod-name> -o jsonpath='{.spec.terminationGracePeriodSeconds}'`	Checks the configured grace period that may explain a slow termination.
`kubectl delete pod <pod-name> --grace-period=0 --force`	Force-removes a pod stuck in Terminating state after investigation.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[0].state.terminated.exitCode}'`	Checks the exact process exit code of a crashed container.
`kubectl exec <pod-name> -- cat /proc/1/status`	Inspects process 1 status inside the container for signal or state clues right before a crash.
`kubectl get pod <pod-name> -o jsonpath='{.status.containerStatuses[0].restartCount}'`	Confirms the exact restart count to gauge crash frequency.

Command	Description
`kubectl get events --field-selector reason=FailedScheduling --field-selector involvedObject.name=<pod-name>`	Filters scheduling failure events for a specific pending pod.
`kubectl describe pod <pod-name> | grep -i 'insufficient'`	Quickly finds resource insufficiency messages (cpu/memory/pods) blocking scheduling.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].command}'`	Reviews the container's command override, a common cause of immediate exit crashes.
`kubectl get pod <pod-name> -o jsonpath='{.spec.containers[*].args}'`	Reviews container arguments that might be malformed and causing startup failure.
`kubectl run debug-shell --rm -it --image=<same-image> --restart=Never -- sh`	Runs the same image standalone with a shell entrypoint to debug startup logic interactively.

15. Kubectl Debug, Exec & Advanced Diagnostics

Advanced diagnostic commands using ephemeral containers, ksniff-style packet capture, and direct API queries for deep troubleshooting.

20 commands in this category

Command	Description
`kubectl debug -it <pod-name> --image=nicolaka/netshoot --target=<container-name>`	Attaches a feature-rich network debugging container sharing the target's namespaces.
`kubectl debug node/<node-name> -it --image=ubuntu -- chroot /host bash`	Gets a full shell on the underlying node's filesystem for OS-level debugging.
`kubectl exec <pod-name> -- tcpdump -i eth0 -w /tmp/capture.pcap`	Captures network traffic inside a pod for offline packet analysis.
`kubectl cp <pod-name>:/tmp/capture.pcap ./capture.pcap`	Retrieves a captured packet file from a pod for local analysis with Wireshark.
`kubectl exec <pod-name> -- strace -p 1`	Traces system calls of the main process inside a container to debug hangs.
`kubectl exec <pod-name> -- lsof -i`	Lists open network connections and file descriptors inside a container.
`kubectl get --raw /api/v1/namespaces/<ns>/pods/<pod>/log`	Queries raw pod logs directly via the API server REST endpoint.
`kubectl get --raw /apis/apps/v1/namespaces/<ns>/deployments/<name>/status`	Queries raw deployment status directly via the REST API for scripting integrations.
`kubectl proxy &`	Runs a background API proxy for scripted curl-based diagnostics.
`kubectl get pod <pod-name> -o yaml --export 2>/dev/null || kubectl get pod <pod-name> -o yaml`	Retrieves a clean export-style manifest for re-creating a pod elsewhere (export flag deprecated in modern versions).
`kubectl events --for pod/<pod-name>`	Modern subcommand alternative to field-selector event filtering for a specific pod (kubectl 1.27+).
`kubectl debug <pod-name> -it --image=busybox --share-processes --copy-to=debug-copy`	Creates a debug copy sharing the process namespace to inspect a misbehaving main process safely.
`kubectl exec <pod-name> -- /bin/sh -c 'while true; do date; sleep 1; done'`	Runs an inline diagnostic loop inside a container to test ongoing behavior over time.
`kubectl attach <pod-name> -i`	Attaches to the main process stdin/stdout of a running container without starting a new shell.
`kubectl get pod <pod-name> --output=jsonpath-as-json='{.status}'`	Outputs a JSON-formatted (rather than raw jsonpath) view of pod status for easier parsing.
`kubectl api-resources`	Lists all API resource types available in the cluster, useful

Command	Description
	when commands return 'resource not found'.
`kubectl api-versions`	Lists all supported API versions, useful for diagnosing version-mismatch errors in manifests.
`kubectl explain <resource>.spec --recursive`	Recursively prints the full schema of a resource's spec field for manifest debugging.
`kubectl get all -n <namespace>`	Lists all standard resource types in a namespace for a quick full overview.
`kubectl get all -A -o wide | grep -i error`	Greps across all resources cluster-wide for any visible error indicators.

16. Cluster Upgrades & Maintenance

Commands for safely diagnosing and executing cluster version upgrades, component compatibility checks, and maintenance windows.

16 commands in this category

Command	Description
`kubectl version --short`	Quickly checks client/server version skew before planning an upgrade.
`kubeadm upgrade plan`	Shows available upgrade targets and component compatibility for kubeadm clusters.
`kubeadm upgrade apply <version>`	Applies a control plane upgrade to a specified version.
`kubeadm upgrade node`	Upgrades kubelet configuration on a worker node after control plane upgrade.
`kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`	Safely empties a node before upgrading its kubelet or OS packages.
`apt-get update && apt-get install -y kubelet=<version> kubectl=<version>`	Upgrades kubelet/kubectl binaries to a specific pinned version on Debian-based nodes.
`kubectl get nodes -o custom-columns=NAME:.metadata.name,VERSION:.status.nodeInfo.kubeletVersion`	Audits kubelet versions across all nodes to spot skew before/after an upgrade.
`kubectl get pods -A -o jsonpath='{range .items[*]}{.spec.containers[*].image} {"\n"}{end}' | sort -u`	Audits all unique container images in use, useful before deprecating an old registry.
`kubectl get crds`	Lists all CustomResourceDefinitions, important to verify compatibility before upgrades.
`kubectl get apiservices | grep False`	Finds any unavailable aggregated API services that could block an upgrade.
`kubectl get deprecations 2>/dev/null || kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis`	Checks for usage of deprecated APIs that may break after an upgrade.
`pluto detect-helm -owide`	Uses the Pluto tool to detect deprecated/removed API usage in Helm releases before upgrading.
`kubectl get events -A --field-selector reason=NodeNotReady --sort-by=.lastTimestamp`	Reviews recent NodeNotReady events that might indicate maintenance-related instability.
`kubectl get poddisruptionbudgets -A`	Lists PodDisruptionBudgets that could block node drains during maintenance.
`kubectl describe pdb <name>`	Shows current vs desired healthy pod counts enforced by a PodDisruptionBudget.
`kubectl get nodes -o jsonpath='{.items[*].status.nodeInfo.osImage}'`	Audits node OS versions before applying OS-level patches during maintenance.

17. Jobs & CronJobs

Commands for diagnosing failed batch jobs, stuck CronJob schedules, and completion issues.

17 commands in this category

Command	Description
`kubectl get jobs`	Lists Jobs in the current namespace with completion status.
`kubectl describe job <name>`	Shows Job conditions, pod template, and related events explaining failures.
`kubectl get pods -l job-name=<name>`	Lists pods created by a specific Job to inspect individual task failures.
`kubectl logs -l job-name=<name>`	Retrieves logs from all pods belonging to a Job.
`kubectl delete job <name>`	Deletes a stuck or completed Job, optionally before re-triggering it.
`kubectl get job <name> -o jsonpath='{.status.failed}'`	Checks how many Job pod attempts have failed.
`kubectl get job <name> -o jsonpath='{.status.succeeded}'`	Checks how many Job pod attempts have succeeded.
`kubectl get job <name> -o jsonpath='{.spec.backoffLimit}'`	Checks the configured retry limit before a Job is marked failed.
`kubectl get cronjobs`	Lists CronJobs and their schedules and last run times.
`kubectl describe cronjob <name>`	Shows CronJob schedule, concurrency policy, and recent job history events.
`kubectl get cronjob <name> -o jsonpath='{.spec.schedule}'`	Checks the exact cron schedule expression configured.
`kubectl get cronjob <name> -o jsonpath='{.spec.concurrencyPolicy}'`	Checks the concurrency policy (Allow/Forbid/Replace), relevant to overlapping run issues.
`kubectl create job --from=cronjob/<name> manual-test-run`	Manually triggers a CronJob's job template immediately for testing, bypassing the schedule.
`kubectl get cronjob <name> -o jsonpath='{.status.lastScheduleTime}'`	Checks when a CronJob was last actually scheduled to run.
`kubectl patch cronjob <name> -p '{"spec":{"suspend":true}}'`	Suspends a CronJob from triggering new runs during investigation.
`kubectl patch cronjob <name> -p '{"spec":{"suspend":false}}'`	Resumes a previously suspended CronJob.
`kubectl get events --field-selector involvedObject.kind=CronJob`	Filters events specific to CronJob scheduling issues.

18. Horizontal & Vertical Autoscaling

Commands for diagnosing HPA/VPA scaling decisions, metric pipeline failures, and scaling event history.

15 commands in this category

Command	Description
`kubectl get hpa -A`	Lists all HorizontalPodAutoscalers across the cluster.
`kubectl describe hpa <name> -n <namespace>`	Shows detailed HPA conditions, current metrics, and recent scaling events.
`kubectl get hpa <name> -o jsonpath='{.status.conditions}'`	Extracts HPA conditions like AbleToScale and ScalingActive.
`kubectl get hpa <name> -o jsonpath='{.spec.metrics}'`	Reviews configured metric sources (CPU, memory, custom, external) driving scaling decisions.
`kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1`	Checks if the custom metrics API is registered and serving data for custom-metric HPAs.
`kubectl get apiservice v1beta1.custom.metrics.k8s.io`	Confirms the custom metrics adapter API service is available.
`kubectl logs -n <namespace> -l app=prometheus-adapter`	Reviews Prometheus adapter logs when custom metric HPAs fail to scale.
`kubectl get events --field-selector involvedObject.kind=HorizontalPodAutoscaler`	Filters events specific to HPA scaling decisions and failures.
`kubectl get hpa <name> -o jsonpath='{.status.desiredReplicas}'`	Checks what replica count the HPA currently wants to scale to.
`kubectl get hpa <name> -o jsonpath='{.status.currentReplicas}'`	Checks the actual current replica count as last observed by the HPA.
`kubectl patch hpa <name> -p '{"spec":{"minReplicas":1}}'`	Adjusts HPA minimum replica bound directly during a scaling incident.
`kubectl get vpa <name> -o jsonpath='{.status.recommendation}'`	Extracts VPA's recommended CPU/memory values for right-sizing a workload.
`kubectl describe vpa <name> -n <namespace>`	Shows full VPA recommendation history and update policy details.
`kubectl get pods -n kube-system -l app=vpa-recommender`	Confirms the VPA recommender component is running when recommendations stop updating.
`kubectl logs -n kube-system -l app=vpa-updater`	Reviews VPA updater logs when automatic pod resource updates aren't applying.

19. Network Policies & Connectivity Isolation

Dedicated commands for isolating whether traffic blockages are caused by NetworkPolicy rules versus other networking layers.

15 commands in this category

Command	Description
`kubectl get networkpolicy -A`	Lists all NetworkPolicies across the cluster to scope the investigation.
`kubectl describe networkpolicy <name> -n <namespace>`	Shows the exact pod selectors and ingress/egress rules of a policy.
`kubectl get networkpolicy <name> -o jsonpath='{.spec.podSelector}'`	Extracts which pods a policy actually applies to.
`kubectl get networkpolicy <name> -o jsonpath='{.spec.ingress}'`	Extracts allowed ingress sources and ports defined by a policy.
`kubectl get networkpolicy <name> -o jsonpath='{.spec.egress}'`	Extracts allowed egress destinations and ports defined by a policy.
`kubectl get pods --show-labels -n <namespace>`	Cross-checks pod labels against a NetworkPolicy's podSelector for mismatches.
`kubectl exec <source-pod> -- curl -m 5 -v <target-service>`	Tests connectivity with a timeout to clearly identify a network-level block versus an app-level failure.
`kubectl delete networkpolicy <name>`	Temporarily removes a policy to confirm it's the actual cause of a connectivity block (test environments only).
`kubectl get pods -n kube-system -l k8s-app=calico-node`	Checks Calico CNI pod health when NetworkPolicy enforcement seems inconsistent.
`kubectl logs -n kube-system -l k8s-app=calico-node`	Reviews Calico node logs for policy programming errors.
`calicoctl get networkpolicy`	Lists NetworkPolicies from Calico's own CLI perspective for deeper CNI-level inspection.
`kubectl get pods -n kube-system -l app=cilium`	Checks Cilium CNI pod health relevant to eBPF-based policy enforcement.
`cilium status`	Shows Cilium agent health and policy enforcement mode (run inside a Cilium pod via exec).
`cilium policy get`	Lists currently loaded Cilium network policies for direct verification.
`kubectl get events --field-selector reason=NetworkNotReady`	Filters events indicating the underlying network plugin isn't ready on a node.

20. Miscellaneous & Advanced Cluster Diagnostics

Additional high-value commands covering object discovery, finalizers, namespace stuck-deletion, and general-purpose troubleshooting utilities.

21 commands in this category

Command	Description
`kubectl get namespaces`	Lists all namespaces and their status (Active/Terminating).
`kubectl get namespace <name> -o jsonpath='{.status.conditions}'`	Checks why a namespace deletion is stuck, often due to remaining finalizers.
`kubectl get namespace <name> -o json | jq '.spec.finalizers'`	Inspects finalizers blocking namespace deletion.
`kubectl patch namespace <name> -p '{"spec":{"finalizers":[]}}' --type=merge`	Force-removes finalizers to unblock a stuck namespace deletion (use with caution).
`kubectl get all --all-namespaces -o wide`	Dumps a full inventory of standard resources across the entire cluster.
`kubectl get crd | grep <keyword>`	Searches installed CustomResourceDefinitions related to a specific operator or feature.
`kubectl get <custom-resource> -A`	Lists instances of a custom resource across all namespaces.
`kubectl describe <custom-resource> <name>`	Shows status and events for a custom resource managed by an operator.
`kubectl get pod <pod-name> -o jsonpath='{.metadata.finalizers}'`	Checks for finalizers preventing a pod from being fully deleted.
`kubectl patch pod <pod-name> -p '{"metadata":{"finalizers":null}}' --type=merge`	Removes finalizers blocking pod deletion in stuck-terminating scenarios.
`kubectl get all -l app=<label> -A`	Finds every resource type tagged with a given application label across namespaces.
`kubectl get events --field-selector involvedObject.apiVersion!=v1`	Filters events for non-core API objects, useful when debugging operator-managed resources.
`kubectl get mutatingwebhookconfigurations, validatingwebhookconfigurations`	Lists both webhook types together to quickly audit all active admission control points.
`kubectl config view`	Displays the current kubeconfig contexts, clusters, and users for verifying the right context is active.
`kubectl config current-context`	Confirms exactly which cluster/context kubectl commands are currently targeting.
`kubectl config get-contexts`	Lists all available contexts to avoid accidentally troubleshooting the wrong cluster.

Command	Description
`kubectl config use-context <context-name>`	Switches active context, a frequent source of 'it's not working' confusion during multi-cluster work.
`kubectl get nodes --context=<context-name>`	Runs a command against a specific context without switching the default.
`kubectl get pods --context=<context> --namespace=<namespace>`	Combines context and namespace flags for one-off cross-cluster checks.
`kubectl version --output=yaml`	Outputs detailed client/server version info in YAML for build metadata comparisons.
`kubectl get pod <pod-name> -o jsonpath='{.metadata.annotations}'`	Reviews all annotations on a pod, which often carry operator or sidecar-injection metadata relevant to failures.

Closing Notes & Credits

Kubernetes troubleshooting is rarely about memorizing every command — it's about knowing which layer of the stack to inspect first: node, pod, network, storage, or control plane. Use this reference to build that instinct, then go deeper into logs and events for the specifics of each incident.

Stay Connected

For video walkthroughs, real-world incident breakdowns, and hands-on DevOps & Cloud tutorials, subscribe to the DevOps Shack YouTube channel and visit devopsshack.com for more guides, courses, and resources.

DevOpsShack

youtube.com/@DevOpsShack • devopsshack.com